

Synthext

1.0

Generated by Doxygen 1.16.1

1 Namespace Index	1
1.1 Namespace List	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Namespace Documentation	9
5.1 synthlib Namespace Reference	9
5.1.1 Detailed Description	11
6 Class Documentation	13
6.1 synthlib::Bounded< T, min, max, initial > Class Template Reference	13
6.1.1 Detailed Description	14
6.1.2 Constructor & Destructor Documentation	14
6.1.2.1 Bounded()	14
6.1.3 Member Function Documentation	14
6.1.3.1 add_saturated()	14
6.1.3.2 sub_saturated()	14
6.1.3.3 value()	15
6.2 synthlib::BpmChange Struct Reference	15
6.2.1 Detailed Description	15
6.3 synthlib::BpmManager Class Reference	15
6.3.1 Detailed Description	16
6.3.2 Constructor & Destructor Documentation	16
6.3.2.1 BpmManager()	16
6.3.3 Member Function Documentation	16
6.3.3.1 change_bpm()	16
6.4 synthlib::Command Class Reference	17
6.4.1 Detailed Description	17
6.4.2 Constructor & Destructor Documentation	18
6.4.2.1 Command() [1/4]	18
6.4.2.2 Command() [2/4]	18
6.4.2.3 Command() [3/4]	18
6.4.2.4 Command() [4/4]	18
6.4.3 Member Function Documentation	19
6.4.3.1 instrument()	19
6.4.3.2 kind()	19
6.4.3.3 note()	19

6.5 synthlib::Compiler Class Reference	20
6.5.1 Detailed Description	20
6.5.2 Member Function Documentation	20
6.5.2.1 compile()	20
6.5.2.2 compile_line()	21
6.5.2.3 compile_to_file()	21
6.6 synthlib::CompilerError Class Reference	22
6.6.1 Detailed Description	22
6.6.2 Member Function Documentation	22
6.6.2.1 kind()	22
6.6.2.2 line()	23
6.7 synthlib::Group Struct Reference	23
6.8 synthlib::Instrument Class Reference	23
6.8.1 Detailed Description	24
6.8.2 Constructor & Destructor Documentation	24
6.8.2.1 Instrument() [1/2]	24
6.8.2.2 Instrument() [2/2]	24
6.8.3 Member Function Documentation	25
6.8.3.1 group_name()	25
6.8.3.2 midi()	25
6.8.3.3 name()	25
6.8.3.4 name_numbered()	25
6.8.3.5 to_int()	26
6.9 synthlib::MidiCreator Class Reference	26
6.9.1 Detailed Description	27
6.9.2 Constructor & Destructor Documentation	27
6.9.2.1 MidiCreator()	27
6.9.3 Member Function Documentation	27
6.9.3.1 change_instrument()	27
6.9.3.2 generate_file()	27
6.9.3.3 goto_track()	28
6.9.3.4 pause_beats()	28
6.9.3.5 pause_ticks()	28
6.9.3.6 play_note()	28
6.9.3.7 set_bpm()	29
6.10 synthlib::Player Class Reference	29
6.10.1 Detailed Description	30
6.10.2 Member Function Documentation	30
6.10.2.1 current_tick()	30
6.10.2.2 is_playing()	30
6.10.2.3 load_midi()	30
6.10.2.4 load_soundfont()	31

6.10.2.5 seek()	31
6.10.2.6 total_ticks()	31
6.11 synthlib::Track Struct Reference	31
6.12 synthlib::VoiceManager Class Reference	32
6.12.1 Detailed Description	32
6.12.2 Member Function Documentation	33
6.12.2.1 get_bpm()	33
6.12.2.2 get_instrument()	33
6.12.2.3 get_octave()	33
6.12.2.4 get_volume()	34
6.12.2.5 override_instrument()	34
6.12.2.6 override_octave()	34
6.12.2.7 override_volume()	35
6.12.2.8 reset_instrument()	35
6.12.2.9 reset_octave()	35
6.12.2.10 reset_voice()	35
6.12.2.11 reset_volume()	36
6.12.2.12 set_bpm()	36
7 File Documentation	37
7.1 bpm_manager.hpp	37
7.2 command.hpp	37
7.3 compiler.hpp	38
7.4 instrument.hpp	39
7.5 midi_creator.hpp	41
7.6 player.hpp	42
7.7 primitives.hpp	42
7.8 voice_manager.hpp	44
Index	45

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

synthlib	This namespace contain all synthlib objects	9
--------------------------	---	---

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

synthlib::Bounded< T, min, max, initial >	13
synthlib::BpmChange	15
synthlib::BpmManager	15
synthlib::Command	17
synthlib::Compiler	20
std::exception	
synthlib::CompilerError	22
synthlib::Group	23
synthlib::Instrument	23
synthlib::MidiCreator	26
synthlib::Player	29
synthlib::Track	31
synthlib::VoiceManager	32

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

synthlib::Bounded< T, min, max, initial >	
Generic class for bounded values	13
synthlib::BpmChange	
Bpm change event	15
synthlib::BpmManager	
This class manages bpm synchronization across voices	15
synthlib::Command	
A synthext command. Some commands have associated values	17
synthlib::Compiler	
This class compiles Synthext text files into MIDI files	20
synthlib::CompilerError	
A Compilation Error	22
synthlib::Group	23
synthlib::Instrument	
A General MIDI instrument	23
synthlib::MidiCreator	
This class is responsible for writing the MIDI file	26
synthlib::Player	
This class plays a MIDI file	29
synthlib::Track	31
synthlib::VoiceManager	
This class is responsible for setting the voices initial parameters	32

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

synthlib/include/synthlib/bpm_manager.hpp	37
synthlib/include/synthlib/command.hpp	37
synthlib/include/synthlib/compiler.hpp	38
synthlib/include/synthlib/instrument.hpp	39
synthlib/include/synthlib/midi_creator.hpp	41
synthlib/include/synthlib/player.hpp	42
synthlib/include/synthlib/primitives.hpp	42
synthlib/include/synthlib/voice_manager.hpp	44

Chapter 5

Namespace Documentation

5.1 synthlib Namespace Reference

this namespace contain all synthlib objects

Classes

- struct [BpmChange](#)
bpm change event
- class [BpmManager](#)
This class manages bpm synchronization across voices.
- class [Command](#)
A synthext command. Some commands have associated values.
- class [CompilerError](#)
A Compilation Error.
- class [Compiler](#)
This class compiles Synthext text files into MIDI files.
- class [Instrument](#)
A General MIDI instrument.
- struct [Track](#)
- class [MidiCreator](#)
This class is responsible for writing the MIDI file.
- class [Player](#)
This class plays a MIDI file.
- class [Bounded](#)
Generic class for bounded values.
- class [VoiceManager](#)
This class is responsible for setting the voices initial parameters.
- struct [Group](#)

Typedefs

- using **Bpm** = [Bounded](#)<int, BPM_MIN, BPM_MAX, BPM_INITIAL>
A range-validated bpm value.
- using **Voiceld** = [Bounded](#)<int, VOICE_MIN, VOICE_MAX, VOICE_INITIAL>
A range-validated voice identifier.
- using **Octave** = [Bounded](#)<int, OCTAVE_MIN, OCTAVE_MAX, OCTAVE_INITIAL>
A range-validated octave value.
- using **Volume** = [Bounded](#)<int, VOLUME_MIN, VOLUME_MAX, VOLUME_INITIAL>
A range-validated volume value.
- using **Channel** = [Bounded](#)<int, CHANNEL_MIN, CHANNEL_MAX, CHANNEL_INITIAL>
A range-validated channel value.
- using **cursor** = string_view::size_type

Enumerations

- enum class [CommandKind](#) : uint8_t {
PlayNote , **Pause** , **PauseOrRepeat** , **Delay** ,
IncreaseBpm , **DecreaseBpm** , **IncreaseOctave** , **DecreaseOctave** ,
DoubleVolume , **ChangeInstrument** , **IncreaseInstrument** }
Types of synthext commands.
- enum class [Midi](#) : uint8_t {
AcousticGrandPiano = 0 , **BrightAcousticPiano** = 1 , **ElectricGrandPiano** = 2 , **HonkytonkPiano** = 3 ,
ElectricPiano1 = 4 , **ElectricPiano2** = 5 , **Harpsichord** = 6 , **Clavinet** = 7 ,
Celesta = 8 , **Glockenspiel** = 9 , **MusicBox** = 10 , **Vibraphone** = 11 ,
Marimba = 12 , **Xylophone** = 13 , **TubularBells** = 14 , **Dulcimer** = 15 ,
DrawbarOrgan = 16 , **PercussiveOrgan** = 17 , **RockOrgan** = 18 , **ChurchOrgan** = 19 ,
ReedOrgan = 20 , **Accordion** = 21 , **Harmonica** = 22 , **TangoAccordion** = 23 ,
AcousticGuitarNylon = 24 , **AcousticGuitarSteel** = 25 , **ElectricGuitarJazz** = 26 , **ElectricGuitarClean** =
27 ,
ElectricGuitarMuted = 28 , **OverdrivenGuitar** = 29 , **DistortionGuitar** = 30 , **GuitarHarmonics** = 31 ,
AcousticBass = 32 , **ElectricBassFinger** = 33 , **ElectricBassPick** = 34 , **FretlessBass** = 35 ,
SlapBass1 = 36 , **SlapBass2** = 37 , **SynthBass1** = 38 , **SynthBass2** = 39 ,
Violin = 40 , **Viola** = 41 , **Cello** = 42 , **Contrabass** = 43 ,
TremoloStrings = 44 , **PizzicatoStrings** = 45 , **OrchestralHarp** = 46 , **Timpani** = 47 ,
StringEnsemble1 = 48 , **StringEnsemble2** = 49 , **SynthStrings1** = 50 , **SynthStrings2** = 51 ,
ChoirAahs = 52 , **VoiceOohs** = 53 , **SynthVoice** = 54 , **OrchestraHit** = 55 ,
Trumpet = 56 , **Trombone** = 57 , **Tuba** = 58 , **MutedTrumpet** = 59 ,
FrenchHorn = 60 , **BrassSection** = 61 , **SynthBrass1** = 62 , **SynthBrass2** = 63 ,
SopranoSax = 64 , **AltoSax** = 65 , **TenorSax** = 66 , **BaritoneSax** = 67 ,
Oboe = 68 , **EnglishHorn** = 69 , **Bassoon** = 70 , **Clarinet** = 71 ,
Piccolo = 72 , **Flute** = 73 , **Recorder** = 74 , **PanFlute** = 75 ,
BlownBottle = 76 , **Shakuhachi** = 77 , **Whistle** = 78 , **Ocarina** = 79 ,
Lead1Square = 80 , **Lead2Sawtooth** = 81 , **Lead3Calliope** = 82 , **Lead4Chiff** = 83 ,
Lead5Charang = 84 , **Lead6Voice** = 85 , **Lead7Fifths** = 86 , **Lead8BassLead** = 87 ,
Pad1NewAge = 88 , **Pad2Warm** = 89 , **Pad3Polysynth** = 90 , **Pad4Choir** = 91 ,
Pad5Bowed = 92 , **Pad6Metallic** = 93 , **Pad7Halo** = 94 , **Pad8Sweep** = 95 ,
FX1Rain = 96 , **FX2Soundtrack** = 97 , **FX3Crystal** = 98 , **FX4Atmosphere** = 99 ,
FX5Brightness = 100 , **FX6Goblins** = 101 , **FX7Echoes** = 102 , **FX8SciFi** = 103 ,
Sitar = 104 , **Banjo** = 105 , **Shamisen** = 106 , **Koto** = 107 ,
Kalimba = 108 , **Bagpipe** = 109 , **Fiddle** = 110 , **Shanai** = 111 ,
TinkleBell = 112 , **Agogo** = 113 , **SteelDrums** = 114 , **Woodblock** = 115 ,
TaikoDrum = 116 , **MelodicTom** = 117 , **SynthDrum** = 118 , **ReverseCymbal** = 119 ,
GuitarFretNoise = 120 , **BreathNoise** = 121 , **Seashore** = 122 , **BirdTweet** = 123 ,
TelephoneRing = 124 , **Helicopter** = 125 , **Applause** = 126 , **Gunshot** = 127 }

A General MIDI instrument value.

- enum class [Note](#) : uint8_t {
C = 0 , **Cs** = 1 , **Db** = 1 , **D** = 2 ,
Ds = 3 , **Eb** = 3 , **E** = 4 , **F** = 5 ,
Fs = 6 , **Gb** = 6 , **G** = 7 , **Gs** = 8 ,
Ab = 8 , **A** = 9 , **As** = 10 , **Bb** = 10 ,
B = 11 }

Class representing musical notes.

Functions

- bool **operator==** (const [Command](#) &lhs, const [Command](#) &rhs)
- bool **cmp** ([BpmChange](#) &a, [BpmChange](#) &b)
- Compiler::Mapping **build_map** ()
- std::vector< std::string > **get_lines** (std::string &source)
separates a string into lines without the newline
- int **voice2track** (int voice)
- void **create_midi_voice** ([MidiCreator](#) &creator, const [VoiceManager](#) ¶ms, [BpmManager](#) &bpm_man, Compiler::Voiceline &voice, [Voiceld](#) voice_id, int track_num)
- std::vector< uint8_t > **create_midi** (const [VoiceManager](#) ¶ms, std::vector< Compiler::Voiceline > voices)
Transforms the list of voice lines into the midi file.
- void **varlen** (uint32_t number, std::vector< uint8_t > &buf)
- void **write_32bit** (uint32_t number, std::vector< uint8_t > &buf)
Writes a 32bit number in big-endian.
- uint8_t **make_status** (uint8_t event, [Channel](#) channel)
- uint8_t **note2midi** ([Note](#) note, [Octave](#) octave)
converts a note + octave into a MIDI note from 0 to 127
- void **midi_event** (uint32_t delta_time, uint8_t event, [Channel](#) channel, uint8_t data1, uint8_t data2, std::vector< uint8_t > &buf)
- void **midi_event** (uint32_t delta_time, uint8_t event, [Channel](#) channel, uint8_t data1, std::vector< uint8_t > &buf)
- void **end_of_track** (uint32_t delta_time, std::vector< uint8_t > &buf)
- uint32_t **bpm2tempo** ([Bpm](#) bpm)
- void **set_tempo** (uint32_t delta_time, [Bpm](#) bpm, std::vector< uint8_t > &buf)
- void **note_on** (uint32_t delta_time, [Channel](#) channel, [Note](#) note, [Octave](#) octave, [Volume](#) velocity, std::vector< uint8_t > &buf)
- void **note_off** (uint32_t delta_time, [Channel](#) channel, [Note](#) note, [Octave](#) octave, [Volume](#) velocity, std::vector< uint8_t > &buf)
- void **all_notes_off** (uint32_t delta_time, [Channel](#) channel, std::vector< uint8_t > &buf)
- void **prog_change** (uint32_t delta_time, [Channel](#) channel, [Instrument](#) instr, std::vector< uint8_t > &buf)
- [Volume](#) **default_volume** ([Voiceld](#) voice)
- [Octave](#) **default_octave** ([Voiceld](#) voice)
- [Instrument](#) **default_instrument** ([Voiceld](#) voice)

Variables

- constexpr int **GROUPS_LEN** = 16
- const uint8_t **NOTE_OFF** = 0x8
- const uint8_t **NOTE_ON** = 0x9
- const uint8_t **PROG_CHANGE** = 0xC
- const uint8_t **CHANNEL_MODE** = 0xB

5.1.1 Detailed Description

this namespace contain all synthlib objects

Chapter 6

Class Documentation

6.1 synthlib::Bounded< T, min, max, initial > Class Template Reference

Generic class for bounded values.

```
#include <primitives.hpp>
```

Public Member Functions

- [Bounded](#) (T [value](#))
Validates and creates the bounded object.
- Bounded< T, min, max, initial > [add_saturated](#) (T [value](#))
Add a value and clamp result inside bounds.
- Bounded< T, min, max, initial > [sub_saturated](#) (T [value](#))
Subtract a value and clamp result inside bounds.
- T [value](#) () const
Returns the inner value.

Static Public Attributes

- static constexpr T **MIN** = min
- static constexpr T **MAX** = max
- static constexpr T **DEFAULT** = initial

Private Attributes

- T [_value](#)

Friends

- bool **operator**< (const Bounded< T, min, max, initial > &lhs, const Bounded< T, min, max, initial > &rhs)
- bool **operator**> (const Bounded< T, min, max, initial > &lhs, const Bounded< T, min, max, initial > &rhs)
- bool **operator**<= (const Bounded< T, min, max, initial > &lhs, const Bounded< T, min, max, initial > &rhs)
- bool **operator**>= (const Bounded< T, min, max, initial > &lhs, const Bounded< T, min, max, initial > &rhs)
- bool **operator**== (const Bounded< T, min, max, initial > &lhs, const Bounded< T, min, max, initial > &rhs)
- bool **operator**!= (const Bounded< T, min, max, initial > &lhs, const Bounded< T, min, max, initial > &rhs)

6.1.1 Detailed Description

```
template<typename T, T min, T max, T initial>
class synthlib::Bounded< T, min, max, initial >
```

Generic class for bounded values.

6.1.2 Constructor & Destructor Documentation

6.1.2.1 Bounded()

```
template<typename T, T min, T max, T initial>
synthlib::Bounded< T, min, max, initial >::Bounded (
    T value) [inline]
```

Validates and creates the bounded object.

Parameters

<i>value</i>	the value to create the object with
--------------	-------------------------------------

6.1.3 Member Function Documentation

6.1.3.1 add_saturated()

```
template<typename T, T min, T max, T initial>
Bounded< T, min, max, initial > synthlib::Bounded< T, min, max, initial >::add_saturated (
    T value) [inline]
```

Add a value and clamp result inside bounds.

Parameters

<i>value</i>	value to add
--------------	--------------

6.1.3.2 sub_saturated()

```
template<typename T, T min, T max, T initial>
Bounded< T, min, max, initial > synthlib::Bounded< T, min, max, initial >::sub_saturated (
    T value) [inline]
```

Subtract a value and clamp result inside bounds.

Parameters

<i>value</i>	value to subtract
--------------	-------------------

6.1.3.3 value()

```
template<typename T, T min, T max, T initial>
T synthlib::Bounded< T, min, max, initial >::value () const [inline], [nodiscard]
```

Returns the inner value.

Returns

the inner value

The documentation for this class was generated from the following file:

- synthlib/include/synthlib/primitives.hpp

6.2 synthlib::BpmChange Struct Reference

bpm change event

```
#include <bpm_manager.hpp>
```

Public Attributes

- uint32_t **beats**
absolute time in beats
- int **bpm_delta**
increase/decrease in bpm

6.2.1 Detailed Description

bpm change event

The documentation for this struct was generated from the following file:

- synthlib/include/synthlib/bpm_manager.hpp

6.3 synthlib::BpmManager Class Reference

This class manages bpm synchronization across voices.

```
#include <bpm_manager.hpp>
```

Public Member Functions

- [BpmManager](#) ([Bpm](#) bpm)
Creates a [BpmManager](#) with the starting bpm.
- void [change_bpm](#) (uint32_t beats, int delta)
Issues a bpm change in absolute time.
- void [write_bpm_track](#) ([MidiCreator](#) &creator, int track_n)
Writes all bpm changes to the track, synchronized and ordered.

Private Attributes

- [Bpm](#) initial_bpm
- std::vector< [BpmChange](#) > changes

6.3.1 Detailed Description

This class manages bpm synchronization across voices.

The MIDI spec doesn't have a "increase bpm" command, only "set bpm", so voices have to know the current bpm to increase it and set to the new bpm. This requires bpm updates to be ordered according to elapsed time.

6.3.2 Constructor & Destructor Documentation

6.3.2.1 BpmManager()

```
synthlib::BpmManager::BpmManager (
    Bpm bpm) [inline]
```

Creates a [BpmManager](#) with the starting bpm.

Parameters

<i>bpm</i>	initial bpm
------------	-------------

6.3.3 Member Function Documentation

6.3.3.1 change_bpm()

```
void synthlib::BpmManager::change_bpm (
    uint32_t beats,
    int delta)
```

Issues a bpm change in absolute time.

Parameters

<i>beats</i>	the song beat in which the change occurs
<i>delta</i>	the value to add to the burrent bpm

The documentation for this class was generated from the following files:

- synthlib/include/synthlib/bpm_manager.hpp
- synthlib/src/bpm_manager.cpp

6.4 synthlib::Command Class Reference

A synthext command. Some commands have associated values.

```
#include <command.hpp>
```

Public Member Functions

- **Command** ()
The default command is pause or repeat, which is issued for the remaining characters.
- **Command** ([Note note](#))
Create a note playing command.
- **Command** ([Instrument instr](#))
Create an instrument change command.
- **Command** ([CommandKind kind](#), int value)
Create a command of type with associated integer value.
- **Command** ([CommandKind kind](#))
Create a command of the given type.
- **CommandKind kind** () const
Return the type of this command.
- **Note note** () const
Return the note if this command is of type PlayNote.
- int **amount** () const
Value used for certain command types.
- **Instrument instrument** () const
Return the target instrument if this command is of type ChangeInstrument.

Private Attributes

- [CommandKind](#) **_kind**
- [Note](#) **_note** = Note::A
- int **_amount** = 0
- [Instrument](#) **_instr** = [Instrument](#)(Midi::AcousticGrandPiano)

6.4.1 Detailed Description

A synthext command. Some commands have associated values.

6.4.2 Constructor & Destructor Documentation

6.4.2.1 Command() [1/4]

```
synthlib::Command::Command (  
    Note note) [inline]
```

Create a note playing command.

Parameters

<i>note</i>	the note to play
-------------	------------------

6.4.2.2 Command() [2/4]

```
synthlib::Command::Command (  
    Instrument instr) [inline]
```

Create an instrument change command.

Parameters

<i>instr</i>	the instrument to change to
--------------	-----------------------------

6.4.2.3 Command() [3/4]

```
synthlib::Command::Command (  
    CommandKind kind,  
    int value) [inline]
```

Create a command of type with associated integer value.

Parameters

<i>kind</i>	the type of command to create
<i>value</i>	the associated value

6.4.2.4 Command() [4/4]

```
synthlib::Command::Command (  
    CommandKind kind) [inline]
```

Create a command of the given type.

Parameters

<i>kind</i>	the type of command to create
-------------	-------------------------------

6.4.3 Member Function Documentation

6.4.3.1 instrument()

```
Instrument synthlib::Command::instrument () const [inline]
```

Return the target instrument if this command is of type ChangeInstrument.

Returns

this command's instrument

Exceptions

<i>std::runtime_error</i>	if this command's type is not ChangeInstrument
---------------------------	--

6.4.3.2 kind()

```
CommandKind synthlib::Command::kind () const [inline]
```

Return the type of this command.

Returns

this command's type

6.4.3.3 note()

```
Note synthlib::Command::note () const [inline]
```

Return the note if this command is of type PlayNote.

Returns

this command's note

Exceptions

<i>std::runtime_error</i>	if the command's type is not PlayNote
---------------------------	---------------------------------------

The documentation for this class was generated from the following file:

- synthlib/include/synthlib/command.hpp

6.5 synthlib::Compiler Class Reference

This class compiles Synthtext text files into MIDI files.

```
#include <compiler.hpp>
```

Public Types

- using **Mapping** = std::array<Command, 256>
- using **Voiceline** = std::vector<Command>

Public Member Functions

- **Compiler** ()
Creates a new compiler.
- std::vector< uint8_t > **compile** (const VoiceManager &voice_params, std::string source) const
Compiles Synthtext source into a MIDI file.
- void **compile_to_file** (const VoiceManager &voice_params, std::string source, std::string filename) const
Compiles Synthtext source and writes to a MIDI file.
- Voiceline **compile_line** (const std::string &line) const
Compiles a Synthtext voice into a sequece of commands.

Private Attributes

- Mapping **map**
The mapping from character -> Command.

6.5.1 Detailed Description

This class compiles Synthtext text files into MIDI files.

The compiler configuration is done through the VoiceParams class.

6.5.2 Member Function Documentation

6.5.2.1 compile()

```
std::vector< uint8_t > synthlib::Compiler::compile (
    const VoiceManager & voice_params,
    std::string source) const
```

Compiles Synthtext source into a MIDI file.

Parameters

<i>voice_params</i>	the voices configuration
---------------------	--------------------------

<i>source</i>	the Synthtext language source text
---------------	------------------------------------

Returns

the MIDI file as a byte vector

6.5.2.2 compile_line()

```
Compiler::Voiceline synthlib::Compiler::compile_line (  
    const std::string & line) const
```

Compiles a Synthtext voice into a sequece of commands.

Compiles a single line into a voice.

Parameters

<i>line</i>	the voice line
-------------	----------------

Returns

A compiled vector of commands

6.5.2.3 compile_to_file()

```
void synthlib::Compiler::compile_to_file (  
    const VoiceManager & voice_params,  
    std::string source,  
    std::string filename) const
```

Compiles Synthtext source and writes to a MIDI file.

Parameters

<i>voice_params</i>	the voices configuration
<i>source</i>	the Synthtext language source text
<i>filename</i>	the filename to write the MIDI to

The documentation for this class was generated from the following files:

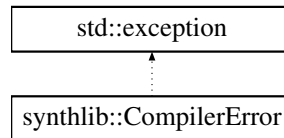
- synthlib/include/synthlib/compiler.hpp
- synthlib/src/compiler.cpp

6.6 synthlib::CompilerError Class Reference

A Compilation Error.

```
#include <compiler.hpp>
```

Inheritance diagram for synthlib::CompilerError:



Public Types

- enum class **Kind** : uint8_t { **UnclosedDelay** , **InvalidDelay** }
- The type of compilation error.*

Public Member Functions

- CompilerError** (**Kind** kind, int line)
Creates a new error with the given kind on the given line.
- const char * **what** () const noexcept override
- Kind** kind () const
- int line () const

Private Member Functions

- std::string **gen_msg** () const

Private Attributes

- Kind** _kind
- int _line
- std::string _msg

6.6.1 Detailed Description

A Compilation Error.

6.6.2 Member Function Documentation

6.6.2.1 kind()

```
Kind synthlib::CompilerError::kind () const [inline]
```

Returns

the type of error

6.6.2.2 line()

```
int synthlib::CompilerError::line () const [inline]
```

Returns

the line of the error

The documentation for this class was generated from the following files:

- synthlib/include/synthlib/compiler.hpp
- synthlib/src/compiler.cpp

6.7 synthlib::Group Struct Reference

Public Attributes

- int **min**
- int **max**
- const char * **name**

The documentation for this struct was generated from the following file:

- synthlib/src/instrument.cpp

6.8 synthlib::Instrument Class Reference

A General MIDI instrument.

```
#include <instrument.hpp>
```

Public Member Functions

- **Instrument** ()
Creates a new General MIDI instrument holding the GM #0 instrument.
- **Instrument** (Midi instrument)
Creates a new General MIDI instrument.
- **Instrument** (uint8_t midi)
Creates a new General MIDI instrument from an integer if it's in the range [MIDI_MIN; MIDI_MAX].
- **Midi midi** () const
- **uint8_t to_int** () const
- const char * **name** () const
- const char * **name_numbered** () const
- const char * **group_name** () const

Static Public Attributes

- static constexpr int **MIDI_MIN** = 0
Minimum GM value.
- static constexpr int **MIDI_MAX** = 127
Maximum GM value.

Private Attributes

- [Midi](#) instr

6.8.1 Detailed Description

A General MIDI instrument.

6.8.2 Constructor & Destructor Documentation

6.8.2.1 Instrument() [1/2]

```
synthlib::Instrument::Instrument (
    Midi instrument) [inline]
```

Creates a new General MIDI instrument.

Parameters

<i>instrument</i>	The MIDI instrument
-------------------	---------------------

Returns

an [Instrument](#) object

6.8.2.2 Instrument() [2/2]

```
synthlib::Instrument::Instrument (
    uint8_t midi) [inline]
```

Creates a new General MIDI instrument from an integer if it's in the range [MIDI_MIN; MIDI_MAX].

Parameters

<i>midi</i>	integer representing a MIDI value
-------------	-----------------------------------

Returns

an [Instrument](#) object

Exceptions

<code>std::invalid_argument</code>	if integer out of range
------------------------------------	-------------------------

6.8.3 Member Function Documentation

6.8.3.1 group_name()

```
const char * synthlib::Instrument::group_name () const [nodiscard]
```

Returns

The instrument category's name

6.8.3.2 midi()

```
Midi synthlib::Instrument::midi () const [inline], [nodiscard]
```

Returns

The MIDI instrument

6.8.3.3 name()

```
const char * synthlib::Instrument::name () const [nodiscard]
```

Returns

The instrument name

6.8.3.4 name_numbered()

```
const char * synthlib::Instrument::name_numbered () const [nodiscard]
```

Returns

The instrument number followed by its name

6.8.3.5 to_int()

```
uint8_t synthlib::Instrument::to_int () const [inline], [nodiscard]
```

Returns

The integer MIDI value

The documentation for this class was generated from the following files:

- synthlib/include/synthlib/instrument.hpp
- synthlib/src/instrument.cpp

6.9 synthlib::MidiCreator Class Reference

This class is responsible for writing the MIDI file.

```
#include <midi_creator.hpp>
```

Public Member Functions

- [MidiCreator](#) (int ticks_per_beat=1)
Creates a [MidiCreator](#) with the amount of MIDI delta time ticks corresponding to one beat.
- void [goto_track](#) (int track_num)
Set the writer to the specified track.
- void [play_note](#) ([Channel](#) channel, [Note](#) note, [Octave](#) octave, [Volume](#) volume)
Plays the note for one beat in the current track.
- void [pause_beats](#) (uint32_t n=1)
Plays n beats of silence.
- void [pause_ticks](#) (uint32_t n)
Pauses for n ticks.
- void [change_instrument](#) ([Channel](#) channel, [Instrument](#) instr)
Changes the track's instrument.
- void [set_bpm](#) ([Bpm](#) bpm)
Issues a Tempo Change event.
- std::vector< uint8_t > [generate_file](#) ()
Generates the MIDI file.

Private Member Functions

- void [write_header](#) (std::vector< uint8_t > &buf)
- void [write_track](#) (std::vector< uint8_t > &buf, int track)

Private Attributes

- std::vector< [Track](#) > **tracks**
- int **curr_track** = -1
- int **beat_ticks**

6.9.1 Detailed Description

This class is responsible for writing the MIDI file.

6.9.2 Constructor & Destructor Documentation

6.9.2.1 MidiCreator()

```
synthlib::MidiCreator::MidiCreator (  
    int ticks_per_beat = 1) [inline]
```

Creates a [MidiCreator](#) with the amount of MIDI delta time ticks corresponding to one beat.

Parameters

<i>ticks_per_beat</i>	number of delta time ticks per beat
-----------------------	-------------------------------------

6.9.3 Member Function Documentation

6.9.3.1 change_instrument()

```
void synthlib::MidiCreator::change_instrument (  
    Channel channel,  
    Instrument instr)
```

Changes the track's instrument.

Parameters

<i>channel</i>	the channel to change the instrument
<i>instr</i>	the target instrument

6.9.3.2 generate_file()

```
std::vector< uint8_t > synthlib::MidiCreator::generate_file ()
```

Generates the MIDI file.

Returns

a vector of bytes containing the generated MIDI file

6.9.3.3 goto_track()

```
void synthlib::MidiCreator::goto_track (
    int track_num)
```

Set the writer to the specified track.

Parameters

<i>track_num</i>	track to enter
------------------	----------------

6.9.3.4 pause_beats()

```
void synthlib::MidiCreator::pause_beats (
    uint32_t n = 1)
```

Plays n beats of silence.

Parameters

<i>n</i>	number of beats
----------	-----------------

6.9.3.5 pause_ticks()

```
void synthlib::MidiCreator::pause_ticks (
    uint32_t n)
```

Pauses for n ticks.

Parameters

<i>n</i>	number of ticks
----------	-----------------

6.9.3.6 play_note()

```
void synthlib::MidiCreator::play_note (
    Channel channel,
    Note note,
    Octave octave,
    Volume volume)
```

Plays the note for one beat in the current track.

Parameters

<i>channel</i>	which channel to play the note
<i>note</i>	the note to play

<i>octave</i>	the note's octave
<i>the</i>	note's volume

6.9.3.7 set_bpm()

```
void synthlib::MidiCreator::set_bpm (
    Bpm bpm)
```

Issues a Tempo Change event.

Parameters

<i>bpm</i>	the target bpm
------------	----------------

The documentation for this class was generated from the following files:

- synthlib/include/synthlib/midi_creator.hpp
- synthlib/src/midi_creator.cpp

6.10 synthlib::Player Class Reference

This class plays a MIDI file.

```
#include <player.hpp>
```

Public Member Functions

- void [load_soundfont](#) (const std::string &file)
Load a soundfont file.
- void [load_midi](#) (const std::vector< uint8_t > &midifile)
Load a MIDI file from memory to play.
- void **play** ()
Start/resume playing the midi file.
- void **stop** ()
Stop playing the midi file.
- void [seek](#) (int ticks)
Seek in the current playing file in ticks.
- int [total_ticks](#) ()
Get the file length in ticks.
- int [current_tick](#) ()
Get the current tick in the file.
- bool [is_playing](#) ()
Returns true if the player is playing a song.
- void **join** ()
Wait for player to finish playing.

Private Attributes

- `std::vector< uint8_t > midi_file`
- `fluid_settings_t * settings = nullptr`
- `fluid_synth_t * synth = nullptr`
- `fluid_player_t * player = nullptr`
- `fluid_audio_driver_t * audio_driver = nullptr`

6.10.1 Detailed Description

This class plays a MIDI file.

6.10.2 Member Function Documentation

6.10.2.1 current_tick()

```
int synthlib::Player::current_tick ()
```

Get the current tick in the file.

Returns

the current tick

6.10.2.2 is_playing()

```
bool synthlib::Player::is_playing ()
```

Returns true if the player is playing a song.

Returns

if the player is currently playing a song

6.10.2.3 load_midi()

```
void synthlib::Player::load_midi (
    const std::vector< uint8_t > & midifile)
```

Load a MIDI file from memory to play.

Parameters

<i>midifile</i>	the MIDI file
-----------------	---------------

6.10.2.4 load_soundfont()

```
void synthlib::Player::load_soundfont (
    const std::string & file)
```

Load a soundfont file.

Parameters

<i>file</i>	the file to load.
-------------	-------------------

6.10.2.5 seek()

```
void synthlib::Player::seek (
    int ticks)
```

Seek in the current playing file in ticks.

Parameters

<i>ticks</i>	the file position in ticks
--------------	----------------------------

6.10.2.6 total_ticks()

```
int synthlib::Player::total_ticks ()
```

Get the file length in ticks.

Returns

the total file ticks

The documentation for this class was generated from the following files:

- synthlib/include/synthlib/player.hpp
- synthlib/src/player.cpp

6.11 synthlib::Track Struct Reference

Public Attributes

- std::vector< uint8_t > **buf**
- uint32_t **delta** = 0

The documentation for this struct was generated from the following file:

- synthlib/include/synthlib/midi_creator.hpp

6.12 synthlib::VoiceManager Class Reference

This class is responsible for setting the voices initial parameters.

```
#include <voice_manager.hpp>
```

Public Member Functions

- void [set_bpm](#) ([Bpm](#) bpm)
Sets the starting BPM.
- void [override_volume](#) ([Voiceld](#) voice, [Volume](#) volume)
Overrides a voice's initial volume.
- void [override_octave](#) ([Voiceld](#) voice, [Octave](#) octave)
Overrides a voice's initial octave.
- void [override_instrument](#) ([Voiceld](#) voice, [Instrument](#) instr)
Overrides a voice's initial instrument.
- void [reset_volume](#) ([Voiceld](#) voice)
Resets a voice's initial volume.
- void [reset_octave](#) ([Voiceld](#) voice)
Resets a voice's initial octave.
- void [reset_instrument](#) ([Voiceld](#) voice)
Resets a voice's initial instrument.
- void [reset_voice](#) ([Voiceld](#) voice)
Resets all parameters of the voice.
- [Volume](#) [get_volume](#) ([Voiceld](#) voice) const
Gets the voice's initial volume.
- [Octave](#) [get_octave](#) ([Voiceld](#) voice) const
Gets the voice's initial octave.
- [Instrument](#) [get_instrument](#) ([Voiceld](#) voice) const
Gets the voice's initial instrument.
- [Bpm](#) [get_bpm](#) () const
Gets the starting bpm.

Private Attributes

- std::map< [Voiceld](#), [Volume](#) > **volume_overrides**
- std::map< [Voiceld](#), [Octave](#) > **octave_overrides**
- std::map< [Voiceld](#), [Instrument](#) > **instrument_overrides**
- [Bpm](#) **_bpm**

6.12.1 Detailed Description

This class is responsible for setting the voices initial parameters.

Voices without any overrides will have default parameters assigned to them according to the specification.

6.12.2 Member Function Documentation

6.12.2.1 get_bpm()

```
Bpm synthlib::VoiceManager::get_bpm () const
```

Gets the starting bpm.

Returns

The starting bpm

6.12.2.2 get_instrument()

```
Instrument synthlib::VoiceManager::get_instrument (  
    VoiceId voice) const
```

Gets the voice's initial instrument.

Parameters

<i>voice</i>	the voice number
--------------	------------------

Returns

The voice's initial instrument

6.12.2.3 get_octave()

```
Octave synthlib::VoiceManager::get_octave (  
    VoiceId voice) const
```

Gets the voice's initial octave.

Parameters

<i>voice</i>	the voice number
--------------	------------------

Returns

The voice's initial octave

6.12.2.4 get_volume()

```
Volume synthlib::VoiceManager::get_volume (
    VoiceId voice) const
```

Gets the voice's initial volume.

Parameters

<i>voice</i>	the voice number
--------------	------------------

Returns

The voice's initial volume

6.12.2.5 override_instrument()

```
void synthlib::VoiceManager::override_instrument (
    VoiceId voice,
    Instrument instr)
```

Overrides a voice's initial instrument.

Parameters

<i>voice</i>	the voice number
<i>instr</i>	the desired instrument

6.12.2.6 override_octave()

```
void synthlib::VoiceManager::override_octave (
    VoiceId voice,
    Octave octave)
```

Overrides a voice's initial octave.

Parameters

<i>voice</i>	the voice number
<i>octave</i>	the desired octave

6.12.2.7 override_volume()

```
void synthlib::VoiceManager::override_volume (
    VoiceId voice,
    Volume volume)
```

Overrides a voice's initial volume.

Parameters

<i>voice</i>	the voice number
<i>volume</i>	the desired volume

6.12.2.8 reset_instrument()

```
void synthlib::VoiceManager::reset_instrument (
    VoiceId voice)
```

Resets a voice's initial instrument.

Parameters

<i>voice</i>	the voice number
--------------	------------------

6.12.2.9 reset_octave()

```
void synthlib::VoiceManager::reset_octave (
    VoiceId voice)
```

Resets a voice's initial octave.

Parameters

<i>voice</i>	the voice number
--------------	------------------

6.12.2.10 reset_voice()

```
void synthlib::VoiceManager::reset_voice (
    VoiceId voice)
```

Resets all parameters of the voice.

Parameters

<i>voice</i>	the voice number
--------------	------------------

6.12.2.11 reset_volume()

```
void synthlib::VoiceManager::reset_volume (  
    VoiceId voice)
```

Resets a voice's initial volume.

Parameters

<i>voice</i>	the voice number
--------------	------------------

6.12.2.12 set_bpm()

```
void synthlib::VoiceManager::set_bpm (  
    Bpm bpm)
```

Sets the starting BPM.

Parameters

<i>bpm</i>	the starting bpm
------------	------------------

The documentation for this class was generated from the following files:

- synthlib/include/synthlib/voice_manager.hpp
- synthlib/src/voice_manager.cpp

Chapter 7

File Documentation

7.1 bpm_manager.hpp

```
00001 #pragma once
00002 #include "synthlib/midi_creator.hpp"
00003 #include "synthlib/primitives.hpp"
00004 #include <vector>
00005
00006 namespace synthlib {
00007
00008     struct BpmChange {
00009         uint32_t beats;
00010         int bpm_delta;
00011     };
00012
00013     class BpmManager {
00014     public:
00015         BpmManager(Bpm bpm) : initial_bpm(bpm) {}
00016         void change_bpm(uint32_t beats, int delta);
00017         void write_bpm_track(MidiCreator &creator, int track_n);
00018     private:
00019         Bpm initial_bpm;
00020         std::vector<BpmChange> changes;
00021     };
00022 } // namespace synthlib
```

7.2 command.hpp

```
00001 #pragma once
00002
00003 #include "synthlib/instrument.hpp"
00004 #include "synthlib/primitives.hpp"
00005 #include <stdint>
00006 #include <stdexcept>
00007 #include <variant>
00008
00009 namespace synthlib {
00010     enum class CommandKind : uint8_t {
00011         PlayNote,
00012         Pause,
00013         PauseOrRepeat,
00014         Delay,
00015         IncreaseBpm,
00016         DecreaseBpm,
00017         IncreaseOctave,
00018         DecreaseOctave,
00019         DoubleVolume,
00020         ChangeInstrument,
00021         IncreaseInstrument
00022     };
00023
00024     class Command {
00025     private:
```

```

00028 CommandKind _kind;
00029 Note _note = Note::A;
00030 int _amount = 0;
00031 Instrument _instr = Instrument(Midi::AcousticGrandPiano);
00032
00033 public:
00034 Command() : _kind(CommandKind::PauseOrRepeat) {}
00035
00036 Command(Note note) : _kind(CommandKind::PlayNote), _note(note) {}
00037 Command(Instrument instr)
00038 : _kind(CommandKind::ChangeInstrument), _instr(instr) {}
00039 Command(CommandKind kind, int value) : _kind(kind), _amount(value) {}
00040 Command(CommandKind kind) : _kind(kind) {}
00041
00042 CommandKind kind() const { return _kind; }
00043 Note note() const {
00044     if (_kind != CommandKind::PlayNote) {
00045         throw std::runtime_error("command kind != PlayNote");
00046     }
00047     return _note;
00048 }
00049 int amount() const { return _amount; }
00050 Instrument instrument() const {
00051     if (_kind != CommandKind::ChangeInstrument) {
00052         throw std::runtime_error("command kind != ChangeInstrument");
00053     }
00054     return _instr;
00055 }
00056 };
00057
00058 bool operator==(const Command &lhs, const Command &rhs);
00059 } // namespace synthlib

```

7.3 compiler.hpp

```

00001 #pragma once
00002 #include "synthlib/command.hpp"
00003 #include "synthlib/voice_manager.hpp"
00004 #include <array>
00005 #include <string>
00006 #include <vector>
00007
00008 namespace synthlib {
00009
00010 class CompilerError : std::exception {
00011 public:
00012     enum class Kind : uint8_t {
00013         UnclosedDelay,
00014         InvalidDelay,
00015     };
00016     CompilerError(Kind kind, int line) : _kind(kind), _line(line) {
00017         _msg = gen_msg();
00018     }
00019     const char *what() const noexcept override { return _msg.c_str(); }
00020
00021     Kind kind() const { return _kind; }
00022     int line() const { return _line; }
00023
00024 private:
00025     Kind _kind;
00026     int _line;
00027     std::string _msg;
00028
00029     std::string gen_msg() const;
00030 };
00031
00032 class Compiler {
00033 public:
00034     using Mapping = std::array<Command, 256>;
00035     using Voiceline = std::vector<Command>;
00036
00037     Compiler();
00038
00039     std::vector<uint8_t> compile(const VoiceManager &voice_params,
00040                                std::string source) const;
00041     void compile_to_file(const VoiceManager &voice_params, std::string source,
00042                        std::string filename) const;
00043     Voiceline compile_line(const std::string &line) const;
00044
00045 private:
00046     Mapping map;

```

```
00070 };
00071
00072 } // namespace synthlib
```

7.4 instrument.hpp

```
00001 #pragma once
00002 #include <stdint>
00003 #include <stdexcept>
00004
00005 namespace synthlib {
00006
00007     enum class Midi : uint8_t {
00008         // Piano:
00009         AcousticGrandPiano = 0,
00010         BrightAcousticPiano = 1,
00011         ElectricGrandPiano = 2,
00012         HonkytonkPiano = 3,
00013         ElectricPiano1 = 4,
00014         ElectricPiano2 = 5,
00015         Harpsichord = 6,
00016         Clavinet = 7,
00017         // Chromatic Percussion:
00018         Celesta = 8,
00019         Glockenspiel = 9,
00020         MusicBox = 10,
00021         Vibraphone = 11,
00022         Marimba = 12,
00023         Xylophone = 13,
00024         TubularBells = 14,
00025         Dulcimer = 15,
00026
00027         // Organ:
00028         DrawbarOrgan = 16,
00029         PercussiveOrgan = 17,
00030         RockOrgan = 18,
00031         ChurchOrgan = 19,
00032         ReedOrgan = 20,
00033         Accordion = 21,
00034         Harmonica = 22,
00035         TangoAccordion = 23,
00036
00037         // Guitar:
00038         AcousticGuitarNylon = 24,
00039         AcousticGuitarSteel = 25,
00040         ElectricGuitarJazz = 26,
00041         ElectricGuitarClean = 27,
00042         ElectricGuitarMuted = 28,
00043         OverdrivenGuitar = 29,
00044         DistortionGuitar = 30,
00045         GuitarHarmonics = 31,
00046
00047         // Bass:
00048         AcousticBass = 32,
00049         ElectricBassFinger = 33,
00050         ElectricBassPick = 34,
00051         FretlessBass = 35,
00052         SlapBass1 = 36,
00053         SlapBass2 = 37,
00054         SynthBass1 = 38,
00055         SynthBass2 = 39,
00056
00057         // Strings:
00058         Violin = 40,
00059         Viola = 41,
00060         Cello = 42,
00061         Contrabass = 43,
00062         TremoloStrings = 44,
00063         PizzicatoStrings = 45,
00064         OrchestralHarp = 46,
00065         Timpani = 47,
00066
00067         // Strings (continued):
00068         StringEnsemble1 = 48,
00069         StringEnsemble2 = 49,
00070         SynthStrings1 = 50,
00071         SynthStrings2 = 51,
00072         ChoirAahs = 52,
00073         VoiceOohs = 53,
00074         SynthVoice = 54,
00075         OrchestraHit = 55,
00076
00077         // Brass:
00078         Trumpet = 56,
00079         Trombone = 57,
00080         Tuba = 58,
00081         Euphonium = 59,
00082         FrenchHorn = 60,
00083         ItalianHorn = 61,
00084         Saxophone = 62,
00085         Baritone = 63,
00086         Bassoon = 64,
00087         Clarinet = 65,
00088         Piccolo = 66,
00089         Flute = 67,
00090         Recorder = 68,
00091         PanFlute = 69,
00092         Whistle = 70,
00093         Glockenspiel = 71,
00094         MusicBox = 72,
00095         Vibraphone = 73,
00096         Marimba = 74,
00097         Xylophone = 75,
00098         TubularBells = 76,
00099         Dulcimer = 77,
00100         Celesta = 78,
00101         Harpsichord = 79,
00102         Clavinet = 80,
00103         ElectricPiano1 = 81,
00104         ElectricPiano2 = 82,
00105         HonkytonkPiano = 83,
00106         ElectricGrandPiano = 84,
00107         BrightAcousticPiano = 85,
00108         AcousticGrandPiano = 86,
00109         AcousticGuitarNylon = 87,
00110         AcousticGuitarSteel = 88,
00111         ElectricGuitarJazz = 89,
00112         ElectricGuitarClean = 90,
00113         ElectricGuitarMuted = 91,
00114         OverdrivenGuitar = 92,
00115         DistortionGuitar = 93,
00116         GuitarHarmonics = 94,
00117         TangoAccordion = 95,
00118         Harmonica = 96,
00119         Accordion = 97,
00120         ReedOrgan = 98,
00121         ChurchOrgan = 99,
00122         RockOrgan = 100,
00123         PercussiveOrgan = 101,
00124         DrawbarOrgan = 102,
00125         Harpsichord = 103,
00126         Clavinet = 104,
00127         ElectricPiano1 = 105,
00128         ElectricPiano2 = 106,
00129         HonkytonkPiano = 107,
00130         ElectricGrandPiano = 108,
00131         BrightAcousticPiano = 109,
00132         AcousticGrandPiano = 110,
00133         AcousticGuitarNylon = 111,
00134         AcousticGuitarSteel = 112,
00135         ElectricGuitarJazz = 113,
00136         ElectricGuitarClean = 114,
00137         ElectricGuitarMuted = 115,
00138         OverdrivenGuitar = 116,
00139         DistortionGuitar = 117,
00140         GuitarHarmonics = 118,
00141         TangoAccordion = 119,
00142         Harmonica = 120,
00143         Accordion = 121,
00144         ReedOrgan = 122,
00145         ChurchOrgan = 123,
00146         RockOrgan = 124,
00147         PercussiveOrgan = 125,
00148         DrawbarOrgan = 126,
00149         Harpsichord = 127,
00150         Clavinet = 128,
00151         ElectricPiano1 = 129,
00152         ElectricPiano2 = 130,
00153         HonkytonkPiano = 131,
00154         ElectricGrandPiano = 132,
00155         BrightAcousticPiano = 133,
00156         AcousticGrandPiano = 134,
00157         AcousticGuitarNylon = 135,
00158         AcousticGuitarSteel = 136,
00159         ElectricGuitarJazz = 137,
00160         ElectricGuitarClean = 138,
00161         ElectricGuitarMuted = 139,
00162         OverdrivenGuitar = 140,
00163         DistortionGuitar = 141,
00164         GuitarHarmonics = 142,
00165         TangoAccordion = 143,
00166         Harmonica = 144,
00167         Accordion = 145,
00168         ReedOrgan = 146,
00169         ChurchOrgan = 147,
00170         RockOrgan = 148,
00171         PercussiveOrgan = 149,
00172         DrawbarOrgan = 150,
00173         Harpsichord = 151,
00174         Clavinet = 152,
00175         ElectricPiano1 = 153,
00176         ElectricPiano2 = 154,
00177         HonkytonkPiano = 155,
00178         ElectricGrandPiano = 156,
00179         BrightAcousticPiano = 157,
00180         AcousticGrandPiano = 158,
00181         AcousticGuitarNylon = 159,
00182         AcousticGuitarSteel = 160,
00183         ElectricGuitarJazz = 161,
00184         ElectricGuitarClean = 162,
00185         ElectricGuitarMuted = 163,
00186         OverdrivenGuitar = 164,
00187         DistortionGuitar = 165,
00188         GuitarHarmonics = 166,
00189         TangoAccordion = 167,
00190         Harmonica = 168,
00191         Accordion = 169,
00192         ReedOrgan = 170,
00193         ChurchOrgan = 171,
00194         RockOrgan = 172,
00195         PercussiveOrgan = 173,
00196         DrawbarOrgan = 174,
00197         Harpsichord = 175,
00198         Clavinet = 176,
00199         ElectricPiano1 = 177,
00200         ElectricPiano2 = 178,
00201         HonkytonkPiano = 179,
00202         ElectricGrandPiano = 180,
00203         BrightAcousticPiano = 181,
00204         AcousticGrandPiano = 182,
00205         AcousticGuitarNylon = 183,
00206         AcousticGuitarSteel = 184,
00207         ElectricGuitarJazz = 185,
00208         ElectricGuitarClean = 186,
00209         ElectricGuitarMuted = 187,
00210         OverdrivenGuitar = 188,
00211         DistortionGuitar = 189,
00212         GuitarHarmonics = 190,
00213         TangoAccordion = 191,
00214         Harmonica = 192,
00215         Accordion = 193,
00216         ReedOrgan = 194,
00217         ChurchOrgan = 195,
00218         RockOrgan = 196,
00219         PercussiveOrgan = 197,
00220         DrawbarOrgan = 198,
00221         Harpsichord = 199,
00222         Clavinet = 200,
00223         ElectricPiano1 = 201,
00224         ElectricPiano2 = 202,
00225         HonkytonkPiano = 203,
00226         ElectricGrandPiano = 204,
00227         BrightAcousticPiano = 205,
00228         AcousticGrandPiano = 206,
00229         AcousticGuitarNylon = 207,
00230         AcousticGuitarSteel = 208,
00231         ElectricGuitarJazz = 209,
00232         ElectricGuitarClean = 210,
00233         ElectricGuitarMuted = 211,
00234         OverdrivenGuitar = 212,
00235         DistortionGuitar = 213,
00236         GuitarHarmonics = 214,
00237         TangoAccordion = 215,
00238         Harmonica = 216,
00239         Accordion = 217,
00240         ReedOrgan = 218,
00241         ChurchOrgan = 219,
00242         RockOrgan = 220,
00243         PercussiveOrgan = 221,
00244         DrawbarOrgan = 222,
00245         Harpsichord = 223,
00246         Clavinet = 224,
00247         ElectricPiano1 = 225,
00248         ElectricPiano2 = 226,
00249         HonkytonkPiano = 227,
00250         ElectricGrandPiano = 228,
00251         BrightAcousticPiano = 229,
00252         AcousticGrandPiano = 230,
00253         AcousticGuitarNylon = 231,
00254         AcousticGuitarSteel = 232,
00255         ElectricGuitarJazz = 233,
00256         ElectricGuitarClean = 234,
00257         ElectricGuitarMuted = 235,
00258         OverdrivenGuitar = 236,
00259         DistortionGuitar = 237,
00260         GuitarHarmonics = 238,
00261         TangoAccordion = 239,
00262         Harmonica = 240,
00263         Accordion = 241,
00264         ReedOrgan = 242,
00265         ChurchOrgan = 243,
00266         RockOrgan = 244,
00267         PercussiveOrgan = 245,
00268         DrawbarOrgan = 246,
00269         Harpsichord = 247,
00270         Clavinet = 248,
00271         ElectricPiano1 = 249,
00272         ElectricPiano2 = 250,
00273         HonkytonkPiano = 251,
00274         ElectricGrandPiano = 252,
00275         BrightAcousticPiano = 253,
00276         AcousticGrandPiano = 254,
00277         AcousticGuitarNylon = 255,
00278         AcousticGuitarSteel = 256,
00279         ElectricGuitarJazz = 257,
00280         ElectricGuitarClean = 258,
00281         ElectricGuitarMuted = 259,
00282         OverdrivenGuitar = 260,
00283         DistortionGuitar = 261,
00284         GuitarHarmonics = 262,
00285         TangoAccordion = 263,
00286         Harmonica = 264,
00287         Accordion = 265,
00288         ReedOrgan = 266,
00289         ChurchOrgan = 267,
00290         RockOrgan = 268,
00291         PercussiveOrgan = 269,
00292         DrawbarOrgan = 270,
00293         Harpsichord = 271,
00294         Clavinet = 272,
00295         ElectricPiano1 = 273,
00296         ElectricPiano2 = 274,
00297         HonkytonkPiano = 275,
00298         ElectricGrandPiano = 276,
00299         BrightAcousticPiano = 277,
00300         AcousticGrandPiano = 278,
00301         AcousticGuitarNylon = 279,
00302         AcousticGuitarSteel = 280,
00303         ElectricGuitarJazz = 281,
00304         ElectricGuitarClean = 282,
00305         ElectricGuitarMuted = 283,
00306         OverdrivenGuitar = 284,
00307         DistortionGuitar = 285,
00308         GuitarHarmonics = 286,
00309         TangoAccordion = 287,
00310         Harmonica = 288,
00311         Accordion = 289,
00312         ReedOrgan = 290,
00313         ChurchOrgan = 291,
00314         RockOrgan = 292,
00315         PercussiveOrgan = 293,
00316         DrawbarOrgan = 294,
00317         Harpsichord = 295,
00318         Clavinet = 296,
00319         ElectricPiano1 = 297,
00320         ElectricPiano2 = 298,
00321         HonkytonkPiano = 299,
00322         ElectricGrandPiano = 300,
00323         BrightAcousticPiano = 301,
00324         AcousticGrandPiano = 302,
00325         AcousticGuitarNylon = 303,
00326         AcousticGuitarSteel = 304,
00327         ElectricGuitarJazz = 305,
00328         ElectricGuitarClean = 306,
00329         ElectricGuitarMuted = 307,
00330         OverdrivenGuitar = 308,
00331         DistortionGuitar = 309,
00332         GuitarHarmonics = 310,
00333         TangoAccordion = 311,
00334         Harmonica = 312,
00335         Accordion = 313,
00336         ReedOrgan = 314,
00337         ChurchOrgan = 315,
00338         RockOrgan = 316,
00339         PercussiveOrgan = 317,
00340         DrawbarOrgan = 318,
00341         Harpsichord = 319,
00342         Clavinet = 320,
00343         ElectricPiano1 = 321,
00344         ElectricPiano2 = 322,
00345         HonkytonkPiano = 323,
00346         ElectricGrandPiano = 324,
00347         BrightAcousticPiano = 325,
00348         AcousticGrandPiano = 326,
00349         AcousticGuitarNylon = 327,
00350         AcousticGuitarSteel = 328,
00351         ElectricGuitarJazz = 329,
00352         ElectricGuitarClean = 330,
00353         ElectricGuitarMuted = 331,
00354         OverdrivenGuitar = 332,
00355         DistortionGuitar = 333,
00356         GuitarHarmonics = 334,
00357         TangoAccordion = 335,
00358         Harmonica = 336,
00359         Accordion = 337,
00360         ReedOrgan = 338,
00361         ChurchOrgan = 339,
00362         RockOrgan = 340,
00363         PercussiveOrgan = 341,
00364         DrawbarOrgan = 342,
00365         Harpsichord = 343,
00366         Clavinet = 344,
00367         ElectricPiano1 = 345,
00368         ElectricPiano2 = 346,
00369         HonkytonkPiano = 347,
00370         ElectricGrandPiano = 348,
00371         BrightAcousticPiano = 349,
00372         AcousticGrandPiano = 350,
00373         AcousticGuitarNylon = 351,
00374         AcousticGuitarSteel = 352,
00375         ElectricGuitarJazz = 353,
00376         ElectricGuitarClean = 354,
00377         ElectricGuitarMuted = 355,
00378         OverdrivenGuitar = 356,
00379         DistortionGuitar = 357,
00380         GuitarHarmonics = 358,
00381         TangoAccordion = 359,
00382         Harmonica = 360,
00383         Accordion = 361,
00384         ReedOrgan = 362,
00385         ChurchOrgan = 363,
00386         RockOrgan = 364,
00387         PercussiveOrgan = 365,
00388         DrawbarOrgan = 366,
00389         Harpsichord = 367,
00390         Clavinet = 368,
00391         ElectricPiano1 = 369,
00392         ElectricPiano2 = 370,
00393         HonkytonkPiano = 371,
00394         ElectricGrandPiano = 372,
00395         BrightAcousticPiano = 373,
00396         AcousticGrandPiano = 374,
00397         AcousticGuitarNylon = 375,
00398         AcousticGuitarSteel = 376,
00399         ElectricGuitarJazz = 377,
00400         ElectricGuitarClean = 378,
00401         ElectricGuitarMuted = 379,
00402         OverdrivenGuitar = 380,
00403         DistortionGuitar = 381,
00404         GuitarHarmonics = 382,
00405         TangoAccordion = 383,
00406         Harmonica = 384,
00407         Accordion = 385,
00408         ReedOrgan = 386,
00409         ChurchOrgan = 387,
00410         RockOrgan = 388,
00411         PercussiveOrgan = 389,
00412         DrawbarOrgan = 390,
00413         Harpsichord = 391,
00414         Clavinet = 392,
00415         ElectricPiano1 = 393,
00416         ElectricPiano2 = 394,
00417         HonkytonkPiano = 395,
00418         ElectricGrandPiano = 396,
00419         BrightAcousticPiano = 397,
00420         AcousticGrandPiano = 398,
00421         AcousticGuitarNylon = 399,
00422         AcousticGuitarSteel = 400,
00423         ElectricGuitarJazz = 401,
00424         ElectricGuitarClean = 402,
00425         ElectricGuitarMuted = 403,
00426         OverdrivenGuitar = 404,
00427         DistortionGuitar = 405,
00428         GuitarHarmonics = 406,
00429         TangoAccordion = 407,
00430         Harmonica = 408,
00431         Accordion = 409,
00432         ReedOrgan = 410,
00433         ChurchOrgan = 411,
00434         RockOrgan = 412,
00435         PercussiveOrgan = 413,
00436         DrawbarOrgan = 414,
00437         Harpsichord = 415,
00438         Clavinet = 416,
00439         ElectricPiano1 = 417,
00440         ElectricPiano2 = 418,
00441         HonkytonkPiano = 419,
00442         ElectricGrandPiano = 420,
00443         BrightAcousticPiano = 421,
00444         AcousticGrandPiano = 422,
00445         AcousticGuitarNylon = 423,
00446         AcousticGuitarSteel = 424,
00447         ElectricGuitarJazz = 425,
00448         ElectricGuitarClean = 426,
00449         ElectricGuitarMuted = 427,
00450         OverdrivenGuitar = 428,
00451         DistortionGuitar = 429,
00452         GuitarHarmonics = 430,
00453         TangoAccordion = 431,
00454         Harmonica = 432,
00455         Accordion = 433,
00456         ReedOrgan = 434,
00457         ChurchOrgan = 435,
00458         RockOrgan = 436,
00459         PercussiveOrgan = 437,
00460         DrawbarOrgan = 438,
00461         Harpsichord = 439,
00462         Clavinet = 440,
00463         ElectricPiano1 = 441,
00464         ElectricPiano2 = 442,
00465         HonkytonkPiano = 443,
00466         ElectricGrandPiano = 444,
00467         BrightAcousticPiano = 445,
00468         AcousticGrandPiano = 446,
00469         AcousticGuitarNylon = 447,
00470         AcousticGuitarSteel = 448,
00471         ElectricGuitarJazz = 449,
00472         ElectricGuitarClean = 450,
00473         ElectricGuitarMuted = 451,
00474         OverdrivenGuitar = 452,
00475         DistortionGuitar = 453,
00476         GuitarHarmonics = 454,
00477         TangoAccordion = 455,
00478         Harmonica = 456,
00479         Accordion = 457,
00480         ReedOrgan = 458,
00481         ChurchOrgan = 459,
00482         RockOrgan = 460,
00483         PercussiveOrgan = 461,
00484         DrawbarOrgan = 462,
00485         Harpsichord = 463,
00486         Clavinet = 464,
00487         ElectricPiano1 = 465,
00488         ElectricPiano2 = 466,
00489         HonkytonkPiano = 467,
00490         ElectricGrandPiano = 468,
00491         BrightAcousticPiano = 469,
00492         AcousticGrandPiano = 470,
00493         AcousticGuitarNylon = 471,
00494         AcousticGuitarSteel = 472,
00495         ElectricGuitarJazz = 473,
00496         ElectricGuitarClean = 474,
00497         ElectricGuitarMuted = 475,
00498         OverdrivenGuitar = 476,
00499         DistortionGuitar = 477,
00500         GuitarHarmonics = 478,
00501         TangoAccordion = 479,
00502         Harmonica = 480,
00503         Accordion = 481,
00504         ReedOrgan = 482,
00505         ChurchOrgan = 483,
00506         RockOrgan = 484,
00507         PercussiveOrgan = 485,
00508         DrawbarOrgan = 486,
00509         Harpsichord = 487,
00510         Clavinet = 488,
00511         ElectricPiano1 = 489,
00512         ElectricPiano2 = 490,
00513         HonkytonkPiano = 491,
00514         ElectricGrandPiano = 492,
00515         BrightAcousticPiano = 493,
00516         AcousticGrandPiano = 494,
00517         AcousticGuitarNylon = 495,
00518         AcousticGuitarSteel = 496,
00519         ElectricGuitarJazz = 497,
00520         ElectricGuitarClean = 498,
00521         ElectricGuitarMuted = 499,
00522         OverdrivenGuitar = 500,
00523         DistortionGuitar = 501,
00524         GuitarHarmonics = 502,
00525         TangoAccordion = 503,
00526         Harmonica = 504,
00527         Accordion = 505,
00528         ReedOrgan = 506,
00529         ChurchOrgan = 507,
00530         RockOrgan = 508,
00531         PercussiveOrgan = 509,
00532         DrawbarOrgan = 510,
00533         Harpsichord = 511,
00534         Clavinet = 512,
00535         ElectricPiano1 = 513,
00536         ElectricPiano2 = 514,
00537         HonkytonkPiano = 515,
00538         ElectricGrandPiano = 516,
00539         BrightAcousticPiano = 517,
00540         AcousticGrandPiano = 518,
00541         AcousticGuitarNylon = 519,
00542         AcousticGuitarSteel = 520,
00543         ElectricGuitarJazz = 521,
00544         ElectricGuitarClean = 522,
00545         ElectricGuitarMuted = 523,
00546         OverdrivenGuitar = 524,
00547         DistortionGuitar = 525,
00548         GuitarHarmonics = 526,
00549         TangoAccordion = 527,
00550         Harmonica = 528,
00551         Accordion = 529,
00552         ReedOrgan = 530,
00553         ChurchOrgan = 531,
00554         RockOrgan = 532,
00555         PercussiveOrgan = 533,
00556         DrawbarOrgan = 534,
00557         Harpsichord = 535,
00558         Clavinet = 536,
00559         ElectricPiano1 = 537,
00560         ElectricPiano2 = 538,
00561         HonkytonkPiano = 539,
00562         ElectricGrandPiano = 540,
00563         BrightAcousticPiano = 541,
00564         AcousticGrandPiano = 542,
00565         AcousticGuitarNylon = 543,
00566         AcousticGuitarSteel = 544,
00567         ElectricGuitarJazz = 545,
00568         ElectricGuitarClean = 546,
00569         ElectricGuitarMuted = 547,
00570         OverdrivenGuitar = 548,
00571         DistortionGuitar = 549,
00572         GuitarHarmonics = 550,
00573         TangoAccordion = 551,
00574         Harmonica = 552,
00575         Accordion = 553,
00576         ReedOrgan = 554,
00577         ChurchOrgan = 555,
00578         RockOrgan = 556,
00579         PercussiveOrgan = 557,
00580         DrawbarOrgan = 558,
00581         Harpsichord = 559,
00582         Clavinet = 560,
00583         ElectricPiano1 = 561,
00584         ElectricPiano2 = 562,
00585         HonkytonkPiano = 563,
00586         ElectricGrandPiano = 564,
00587         BrightAcousticPiano = 565,
00588         AcousticGrandPiano = 566,
00589         AcousticGuitarNylon = 567,
00590         AcousticGuitarSteel = 568,
00591         ElectricGuitarJazz = 569,
00592         ElectricGuitarClean = 570,
00593         ElectricGuitarMuted = 571,
00594         OverdrivenGuitar = 572,
00595         DistortionGuitar = 573,
00596         GuitarHarmonics = 574,
00597         TangoAccordion = 575,
00598         Harmonica = 576,
00599         Accordion = 577,
00600         ReedOrgan = 578,
00601         ChurchOrgan = 579,
00602         RockOrgan = 580,
00603         PercussiveOrgan = 581,
00604         DrawbarOrgan = 582,
00605         Harpsichord = 583,
00606         Clavinet = 584,
00607         ElectricPiano1 = 585,
00608         ElectricPiano2 = 586,
00609         HonkytonkPiano = 587,
00610         ElectricGrandPiano = 588,
00611         BrightAcousticPiano = 589,
00612         AcousticGrandPiano = 590,
00613         AcousticGuitarNylon = 591,
00614         AcousticGuitarSteel = 592,
00615         ElectricGuitarJazz = 593,
00616         ElectricGuitarClean = 594,
00617         ElectricGuitarMuted = 595,
00618         OverdrivenGuitar = 596,
00619         DistortionGuitar = 597,
00620         GuitarHarmonics = 598,
00621         TangoAccordion = 599,
00622         Harmonica = 600,
00623         Accordion = 601,
00624         ReedOrgan = 602,
00625         ChurchOrgan = 603,
00626         RockOrgan = 604,
00627         PercussiveOrgan = 605,
00628         DrawbarOrgan = 606,
00629         Harpsichord = 607,
00630         Clavinet = 608,
00631         ElectricPiano1 = 609,
00632         ElectricPiano2 = 610,
00633         HonkytonkPiano = 611,
00634         ElectricGrandPiano = 612,
00635         BrightAcousticPiano = 613,
00636         AcousticGrandPiano = 614,
00637         AcousticGuitarNylon = 615,
00638         AcousticGuitarSteel = 616,
00639         ElectricGuitarJazz = 617,
00640         ElectricGuitarClean = 618,
00641         ElectricGuitarMuted = 619,
00642         OverdrivenGuitar = 620,
00643         DistortionGuitar = 621,
00644         GuitarHarmonics = 622,
00645         TangoAccordion = 623,
00646         Harmonica = 624,
00647         Accordion = 625,
00648         ReedOrgan = 626,
00649         ChurchOrgan = 627,
00650         RockOrgan = 628,
00651         PercussiveOrgan = 629,
00652         DrawbarOrgan = 630,
00653         Harpsichord = 631,
00654         Clavinet = 632,
00655         ElectricPiano1 = 633,
00656         ElectricPiano2 = 634,
00657         HonkytonkPiano = 635,
00658         ElectricGrandPiano = 636,
00659         BrightAcousticPiano = 637,
00660         AcousticGrandPiano = 638,
00661         AcousticGuitarNylon = 639,
00662         AcousticGuitarSteel = 640,
00663         ElectricGuitarJazz = 641,
00664         ElectricGuitarClean = 642,
00665         ElectricGuitarMuted = 643,
00666         OverdrivenGuitar = 644,
00667         DistortionGuitar = 645,
00668         GuitarHarmonics = 646,
00669         TangoAccordion = 647,
00670         Harmonica = 648,
00671         Accordion = 649,
00672         ReedOrgan = 650,
00673         ChurchOrgan = 651,
00674         RockOrgan = 652,
00675         PercussiveOrgan = 653,
00676         DrawbarOrgan = 654,
00677         Harpsichord = 655,
00678         Clavinet = 656,
00679         ElectricPiano1 = 657,
00680         ElectricPiano2 = 658,
00681         HonkytonkPiano = 659,
00682         ElectricGrandPiano = 660,
00683         BrightAcousticPiano = 661,
00684         AcousticGrandPiano = 662,
00685         AcousticGuitarNylon = 663,
00686         AcousticGuitarSteel = 664,
00687         ElectricGuitarJazz = 665,
00688         ElectricGuitarClean = 666,
00689         ElectricGuitarMuted = 667,
00690         OverdrivenGuitar = 668,
00691         DistortionGuitar = 669,
00692         GuitarHarmonics = 670,
00693         TangoAccordion = 671,
00694         Harmonica = 672,
00695         Accordion = 673,
00696         ReedOrgan = 674,
00697         ChurchOrgan = 675,
00698         RockOrgan = 676,
00699         PercussiveOrgan = 677,
00700         DrawbarOrgan = 678,
00701         Harpsichord = 679,
00702         Clavinet = 680,
00703         ElectricPiano1 = 681,
00704         ElectricPiano2 = 682,
00705         HonkytonkPiano = 683,
00706         ElectricGrandPiano = 684,
00707         BrightAcousticPiano = 685,
00708         AcousticGrandPiano = 686,
00709         AcousticGuitarNylon = 687,
00710         AcousticGuitarSteel = 688,
00711         ElectricGuitarJazz = 689,
00712         ElectricGuitarClean = 690,
00713         ElectricGuitarMuted = 691,
00714         OverdrivenGuitar = 692,
00715         DistortionGuitar = 693,
00716         GuitarHarmonics = 694,
00717         TangoAccordion = 695,
00718         Harmonica = 696,
00719         Accordion = 697,
00720         ReedOrgan = 698,
00721         ChurchOrgan = 699,
00722         RockOrgan = 700,
00723         PercussiveOrgan = 701,
00724         DrawbarOrgan = 702,
00725         Harpsichord = 703,
00726         Clavinet = 704,
00727         ElectricPiano1 = 705,
00728         ElectricPiano2 = 706,
00729         HonkytonkPiano = 707,
00730         ElectricGrandPiano = 708,
00731         BrightAcousticPiano = 709,
00732         AcousticGrandPiano = 710,
00733         AcousticGuitarNylon = 711,
00734         AcousticGuitarSteel = 712,
00735         ElectricGuitarJazz = 713,
00736         ElectricGuitarClean = 714,
00737         ElectricGuitarMuted = 715,
00738         OverdrivenGuitar = 716,
00739         DistortionGuitar = 717,
00740         GuitarHarmonics = 718,
00741         TangoAccordion = 719,
00742         Harmonica = 720,
00743         Accordion = 72
```

```
00079 Trumpet = 56,
00080 Trombone = 57,
00081 Tuba = 58,
00082 MutedTrumpet = 59,
00083 FrenchHorn = 60,
00084 BrassSection = 61,
00085 SynthBrass1 = 62,
00086 SynthBrass2 = 63,
00087
00088 // Reed:
00089 SopranoSax = 64,
00090 AltoSax = 65,
00091 TenorSax = 66,
00092 BaritoneSax = 67,
00093 Oboe = 68,
00094 EnglishHorn = 69,
00095 Bassoon = 70,
00096 Clarinet = 71,
00097
00098 // Pipe:
00099 Piccolo = 72,
00100 Flute = 73,
00101 Recorder = 74,
00102 PanFlute = 75,
00103 BlownBottle = 76,
00104 Shakuhachi = 77,
00105 Whistle = 78,
00106 Ocarina = 79,
00107
00108 // Synth Lead:
00109 Lead1Square = 80,
00110 Lead2Sawtooth = 81,
00111 Lead3Calliope = 82,
00112 Lead4Chiff = 83,
00113 Lead5Charang = 84,
00114 Lead6Voice = 85,
00115 Lead7Fifths = 86,
00116 Lead8BassLead = 87,
00117
00118 // Synth Pad:
00119 Pad1NewAge = 88,
00120 Pad2Warm = 89,
00121 Pad3Polysynth = 90,
00122 Pad4Choir = 91,
00123 Pad5Bowed = 92,
00124 Pad6Metallic = 93,
00125 Pad7Halo = 94,
00126 Pad8Sweep = 95,
00127
00128 // Synth Effects:
00129 FX1Rain = 96,
00130 FX2Soundtrack = 97,
00131 FX3Crystal = 98,
00132 FX4Atmosphere = 99,
00133 FX5Brightness = 100,
00134 FX6Goblins = 101,
00135 FX7Echoes = 102,
00136 FX8SciFi = 103,
00137
00138 // Ethnic:
00139 Sitar = 104,
00140 Banjo = 105,
00141 Shamisen = 106,
00142 Koto = 107,
00143 Kalimba = 108,
00144 Bagpipe = 109,
00145 Fiddle = 110,
00146 Shanai = 111,
00147
00148 // Percussive:
00149 TinkleBell = 112,
00150 Agogo = 113,
00151 SteelDrums = 114,
00152 Woodblock = 115,
00153 TaikoDrum = 116,
00154 MelodicTom = 117,
00155 SynthDrum = 118,
00156
00157 // Sound effects:
00158 ReverseCymbal = 119,
00159 GuitarFretNoise = 120,
00160 BreathNoise = 121,
00161 Seashore = 122,
00162 BirdTweet = 123,
00163 TelephoneRing = 124,
00164 Helicopter = 125,
00165 Applause = 126,
```

```

00166     Gunshot = 127,
00167 };
00168
00170 class Instrument {
00171 public:
00173     static constexpr int MIDI_MIN = 0;
00175     static constexpr int MIDI_MAX = 127;
00176
00179     Instrument() : instr(Midi::AcousticGrandPiano) {}
00180
00184     Instrument(Midi instrument) : instr(instrument) {}
00185
00191     Instrument(uint8_t midi) {
00192         if (midi > MIDI_MAX) {
00193             throw std::invalid_argument("out of range MIDI value");
00194         }
00195         instr = static_cast<Midi>(midi);
00196     }
00197
00199     [[nodiscard]]
00200     Midi midi() const {
00201         return instr;
00202     }
00203
00205     [[nodiscard]]
00206     uint8_t to_int() const {
00207         return static_cast<uint8_t>(instr);
00208     }
00210     [[nodiscard]]
00211     const char *name() const;
00212
00214     [[nodiscard]]
00215     const char *name_numbered() const;
00216
00218     [[nodiscard]]
00219     const char *group_name() const;
00220
00221 private:
00222     Midi instr;
00223 };
00224
00225 } // namespace synthlib

```

7.5 midi_creator.hpp

```

00001 #pragma once
00002
00003 #include "synthlib/instrument.hpp"
00004 #include "synthlib/primitives.hpp"
00005 #include <stdint>
00006 #include <stdexcept>
00007 #include <vector>
00008
00009 namespace synthlib {
00010
00011 struct Track {
00012     std::vector<uint8_t> buf;
00013     uint32_t delta = 0;
00014 };
00015
00017 class MidiCreator {
00018 public:
00022     MidiCreator(int ticks_per_beat = 1) : beat_ticks(ticks_per_beat) {}
00025     void goto_track(int track_num);
00031     void play_note(Channel channel, Note note, Octave octave, Volume volume);
00034     void pause_beats(uint32_t n = 1);
00037     void pause_ticks(uint32_t n);
00041     void change_instrument(Channel channel, Instrument instr);
00044     void set_bpm(Bpm bpm);
00045
00048     std::vector<uint8_t> generate_file();
00049
00050 private:
00051     std::vector<Track> tracks;
00052     int curr_track = -1;
00053     int beat_ticks;
00054
00055     void write_header(std::vector<uint8_t> &buf);
00056     void write_track(std::vector<uint8_t> &buf, int track);
00057 };
00058
00059 } // namespace synthlib

```

7.6 player.hpp

```

00001 #pragma once
00002 #include "fluidsynth.h"
00003 #include <stdint>
00004 #include <fluidsynth/settings.h>
00005 #include <string>
00006 #include <vector>
00007
00008 namespace synthlib {
00009
00011 class Player {
00012 public:
00013     Player();
00014     ~Player();
00017     void load_soundfont(const std::string &file);
00020     void load_midi(const std::vector<uint8_t> &midifile);
00022     void play();
00024     void stop();
00027     void seek(int ticks);
00030     int total_ticks();
00033     int current_tick();
00034
00037     bool is_playing();
00038
00040     void join();
00041
00042 private:
00043     std::vector<uint8_t> midi_file;
00044     fluid_settings_t *settings = nullptr;
00045     fluid_synth_t *synth = nullptr;
00046     fluid_player_t *player = nullptr;
00047     fluid_audio_driver_t *audio_driver = nullptr;
00048 };
00049
00050 } // namespace synthlib

```

7.7 primitives.hpp

```

00001 #pragma once
00002 #include <stdint>
00003 #include <iostream>
00004 #include <stdexcept>
00005
00007 namespace synthlib {
00008     static constexpr int VOICE_MIN = 0;
00009     static constexpr int VOICE_MAX = 15;
00010     static constexpr int VOICE_INITIAL = 0;
00011
00012     static constexpr int CHANNEL_MIN = 0;
00013     static constexpr int CHANNEL_MAX = 15;
00014     static constexpr int CHANNEL_INITIAL = 0;
00015
00016     static constexpr int BPM_MIN = 0;
00017     static constexpr int BPM_MAX = 400;
00018     static constexpr int BPM_INITIAL = 120;
00019
00020     static constexpr int OCTAVE_MIN = 0;
00021     static constexpr int OCTAVE_MAX = 9;
00022     static constexpr int OCTAVE_INITIAL = 0;
00023
00024     static constexpr int VOLUME_MIN = 0;
00025     static constexpr int VOLUME_MAX = 127;
00026     static constexpr int VOLUME_INITIAL = 0;
00027
00029 template <typename T, T min, T max, T initial> class Bounded {
00030 public:
00031     static constexpr T MIN = min;
00032     static constexpr T MAX = max;
00033     static constexpr T DEFAULT = initial;
00034
00035     Bounded() : _value(DEFAULT) {}
00038     Bounded(T value) : _value(value) {
00039         if (value < MIN || value > MAX) {
00040             throw std::invalid_argument("value out of range");
00041         }
00042     }
00043
00046     Bounded<T, min, max, initial> add_saturated(T value) {
00047         T newval = _value + value;
00048         if (newval < MIN) {
00049             return Bounded<T, min, max, initial>(MIN);

```



```

00050     }
00051     if (newval > MAX) {
00052         return Bounded<T, min, max, initial>(MAX);
00053     }
00054     return Bounded<T, min, max, initial>(newval);
00055 }
00056
00057 Bounded<T, min, max, initial> sub_saturated(T value) {
00058     T newval = _value - value;
00059     if (newval < MIN) {
00060         return Bounded(MIN);
00061     }
00062     if (newval > MAX) {
00063         return Bounded(MAX);
00064     }
00065     return Bounded(newval);
00066 }
00067
00068 [[nodiscard]]
00069 T value() const {
00070     return _value;
00071 }
00072
00073 friend bool operator<(const Bounded<T, min, max, initial> &lhs,
00074                     const Bounded<T, min, max, initial> &rhs) {
00075     return lhs.value() < rhs.value();
00076 }
00077
00078 friend bool operator>(const Bounded<T, min, max, initial> &lhs,
00079                     const Bounded<T, min, max, initial> &rhs) {
00080     return lhs.value() > rhs.value();
00081 }
00082
00083 friend bool operator<=(const Bounded<T, min, max, initial> &lhs,
00084                      const Bounded<T, min, max, initial> &rhs) {
00085     return lhs.value() <= rhs.value();
00086 }
00087
00088 friend bool operator>=(const Bounded<T, min, max, initial> &lhs,
00089                      const Bounded<T, min, max, initial> &rhs) {
00090     return lhs.value() >= rhs.value();
00091 }
00092
00093 friend bool operator==(const Bounded<T, min, max, initial> &lhs,
00094                      const Bounded<T, min, max, initial> &rhs) {
00095     return lhs.value() == rhs.value();
00096 }
00097
00098 friend bool operator!=(const Bounded<T, min, max, initial> &lhs,
00099                      const Bounded<T, min, max, initial> &rhs) {
00100     return lhs.value() != rhs.value();
00101 }
00102
00103 private:
00104     T _value;
00105 };
00106
00107 using Bpm = Bounded<int, BPM_MIN, BPM_MAX, BPM_INITIAL>;
00108
00109 using VoiceId = Bounded<int, VOICE_MIN, VOICE_MAX, VOICE_INITIAL>;
00110
00111 using Octave = Bounded<int, OCTAVE_MIN, OCTAVE_MAX, OCTAVE_INITIAL>;
00112
00113 using Volume = Bounded<int, VOLUME_MIN, VOLUME_MAX, VOLUME_INITIAL>;
00114
00115 using Channel = Bounded<int, CHANNEL_MIN, CHANNEL_MAX, CHANNEL_INITIAL>;
00116
00117 enum class Note : uint8_t {
00118     C = 0,
00119     Cs = 1,
00120     Db = 1,
00121     D = 2,
00122     Ds = 3,
00123     Eb = 3,
00124     E = 4,
00125     F = 5,
00126     Fs = 6,
00127     Gb = 6,
00128     G = 7,
00129     Gs = 8,
00130     Ab = 8,
00131     A = 9,
00132     As = 10,
00133     Bb = 10,
00134     B = 11,
00135 };
00136
00137 } // namespace synthlib

```

7.8 voice_manager.hpp

```
00001 #pragma once
00002
00003 #include "synthlib/instrument.hpp"
00004 #include "synthlib/primitives.hpp"
00005 #include <map>
00006
00007 namespace synthlib {
00008
00014 class VoiceManager {
00015 public:
00018     void set_bpm(Bpm bpm);
00022     void override_volume(VoiceId voice, Volume volume);
00023
00027     void override_octave(VoiceId voice, Octave octave);
00028
00032     void override_instrument(VoiceId voice, Instrument instr);
00033
00036     void reset_volume(VoiceId voice);
00037
00040     void reset_octave(VoiceId voice);
00041
00044     void reset_instrument(VoiceId voice);
00045
00048     void reset_voice(VoiceId voice);
00049
00053     Volume get_volume(VoiceId voice) const;
00054
00058     Octave get_octave(VoiceId voice) const;
00059
00063     Instrument get_instrument(VoiceId voice) const;
00064
00067     Bpm get_bpm() const;
00068
00069 private:
00070     std::map<VoiceId, Volume> volume_overrides;
00071     std::map<VoiceId, Octave> octave_overrides;
00072     std::map<VoiceId, Instrument> instrument_overrides;
00073     Bpm _bpm;
00074 };
00075
00076 } // namespace synthlib
```

Index

add_saturated
 synthlib::Bounded< T, min, max, initial >, [14](#)

Bounded
 synthlib::Bounded< T, min, max, initial >, [14](#)

BpmManager
 synthlib::BpmManager, [16](#)

change_bpm
 synthlib::BpmManager, [16](#)

change_instrument
 synthlib::MidiCreator, [27](#)

Command
 synthlib::Command, [18](#)

compile
 synthlib::Compiler, [20](#)

compile_line
 synthlib::Compiler, [21](#)

compile_to_file
 synthlib::Compiler, [21](#)

current_tick
 synthlib::Player, [30](#)

generate_file
 synthlib::MidiCreator, [27](#)

get_bpm
 synthlib::VoiceManager, [33](#)

get_instrument
 synthlib::VoiceManager, [33](#)

get_octave
 synthlib::VoiceManager, [33](#)

get_volume
 synthlib::VoiceManager, [33](#)

goto_track
 synthlib::MidiCreator, [27](#)

group_name
 synthlib::Instrument, [25](#)

Instrument
 synthlib::Instrument, [24](#)

instrument
 synthlib::Command, [19](#)

is_playing
 synthlib::Player, [30](#)

kind
 synthlib::Command, [19](#)
 synthlib::CompilerError, [22](#)

line
 synthlib::CompilerError, [22](#)

load_midi
 synthlib::Player, [30](#)

load_soundfont
 synthlib::Player, [30](#)

midi
 synthlib::Instrument, [25](#)

MidiCreator
 synthlib::MidiCreator, [27](#)

name
 synthlib::Instrument, [25](#)

name_numbered
 synthlib::Instrument, [25](#)

note
 synthlib::Command, [19](#)

override_instrument
 synthlib::VoiceManager, [34](#)

override_octave
 synthlib::VoiceManager, [34](#)

override_volume
 synthlib::VoiceManager, [34](#)

pause_beats
 synthlib::MidiCreator, [28](#)

pause_ticks
 synthlib::MidiCreator, [28](#)

play_note
 synthlib::MidiCreator, [28](#)

reset_instrument
 synthlib::VoiceManager, [35](#)

reset_octave
 synthlib::VoiceManager, [35](#)

reset_voice
 synthlib::VoiceManager, [35](#)

reset_volume
 synthlib::VoiceManager, [35](#)

seek
 synthlib::Player, [31](#)

set_bpm
 synthlib::MidiCreator, [29](#)
 synthlib::VoiceManager, [36](#)

sub_saturated
 synthlib::Bounded< T, min, max, initial >, [14](#)

synthlib, [9](#)
 synthlib/include/synthlib/bpm_manager.hpp, [37](#)
 synthlib/include/synthlib/command.hpp, [37](#)
 synthlib/include/synthlib/compiler.hpp, [38](#)

- synthlib/include/synthlib/instrument.hpp, 39
- synthlib/include/synthlib/midi_creator.hpp, 41
- synthlib/include/synthlib/player.hpp, 42
- synthlib/include/synthlib/primitives.hpp, 42
- synthlib/include/synthlib/voice_manager.hpp, 44
- synthlib::Bounded< T, min, max, initial >, 13
 - add_saturated, 14
 - Bounded, 14
 - sub_saturated, 14
 - value, 14
- synthlib::BpmChange, 15
- synthlib::BpmManager, 15
 - BpmManager, 16
 - change_bpm, 16
- synthlib::Command, 17
 - Command, 18
 - instrument, 19
 - kind, 19
 - note, 19
- synthlib::Compiler, 20
 - compile, 20
 - compile_line, 21
 - compile_to_file, 21
- synthlib::CompilerError, 22
 - kind, 22
 - line, 22
- synthlib::Group, 23
- synthlib::Instrument, 23
 - group_name, 25
 - Instrument, 24
 - midi, 25
 - name, 25
 - name_numbered, 25
 - to_int, 25
- synthlib::MidiCreator, 26
 - change_instrument, 27
 - generate_file, 27
 - goto_track, 27
 - MidiCreator, 27
 - pause_beats, 28
 - pause_ticks, 28
 - play_note, 28
 - set_bpm, 29
- synthlib::Player, 29
 - current_tick, 30
 - is_playing, 30
 - load_midi, 30
 - load_soundfont, 30
 - seek, 31
 - total_ticks, 31
- synthlib::Track, 31
- synthlib::VoiceManager, 32
 - get_bpm, 33
 - get_instrument, 33
 - get_octave, 33
 - get_volume, 33
 - override_instrument, 34
 - override_octave, 34
 - override_volume, 34
 - reset_instrument, 35
 - reset_octave, 35
 - reset_voice, 35
 - reset_volume, 35
 - set_bpm, 36
- to_int
 - synthlib::Instrument, 25
- total_ticks
 - synthlib::Player, 31
- value
 - synthlib::Bounded< T, min, max, initial >, 14